3rd Year – Second Cycle / Option: CyS
IA for Cybersecurity

Lab 01 : Data Preparation, Model Training, and Evaluation for Cybersecurity AI

Dataset: UNSW-NB15 (UNSW_NB15_training-set.csv and UNSW_NB15_testing-set.csv)

Objective

Apply the full machine learning pipeline covered in the chapter (Data Preparation → Model Training → Evaluation) on a real-world cybersecurity dataset. Special attention is given to proper data preparation, prevention of **data leakage**, handling of class imbalance, and comparison between multi-class and binary classification.

Tasks to complete

1. Data Preparation

- Load the two provided CSV files.
- Explore the dataset: shape, data types, missing values, class distribution (label and attack_cat).
- Clean the data (remove unnecessary columns, handle NaN/Inf values).
- Encode categorical variables (proto, service, state).
- Apply feature scaling (StandardScaler and/or MinMaxScaler) **only on the training set**.
- (Optional) Perform simple feature selection.
- Analyze class imbalance.
- Create a validation set from the training data (stratified split recommended).
-

2. Model Training & Classification strategies

- Train at least two models:
  - Logistic Regression (baseline)
  - Random Forest
- Perform a simple hyperparameter search on the validation set.
- Compare training vs validation performance and comment on overfitting / underfitting.
- Use a scikit-learn Pipeline (preprocessing + model) to avoid data leakage.

The student must perform **three distinct experiments** with these models:

Experiment 1 – Multi-class classification

- Use the attack_cat column (10 classes: Normal + 9 attack types).
- Produce the full classification report (per-class performance).

Experiment 2 – Binary classification "Normal vs Attack"

- Use the label column (0 = Normal, 1 = Attack).
- Compare the performance with Experiment 1.

Experiment 3 – Binary classification between two selected classes + class imbalance

- Select two classes (e.g. Exploits vs Generic, DoS vs Fuzzers, or Normal vs a rare class).
- Create the corresponding subset.

- Train the model **without** and **with** imbalance handling (class_weight='balanced' or SMOTE).
- Compare the results.

3. Evaluation

- For each experiment, compute and visualize:
    - Confusion matrix
    - Accuracy
    - Precision, Recall, F1-score (macro and weighted)
    - ROC-AUC (when relevant)
- Present the results in comparative tables.

Deliverables

- A well-structured and commented Jupyter notebook.
- A report summarizing the work and the main findings.
- Performance tables for the three experiments.
- Code must be reproducible (fixed random_state).

Guidelines

- Use only scikit-learn (plus pandas, numpy, matplotlib, seaborn).
- All transformers (scaler, encoder…) must be fitted **exclusively** on the training set.
- Detailed observations and analysis of the results will be discussed during the consultation session.